数学与 CS 问答笔记

answer-note

0101 步骤 0 的评估，以及 ADT 测试中 operation 与 partition 如何分层组织；只给判定与方法，不给练习答案。

October 31, 2026

Contents

1. 0101 步骤 0 评估：AF / RI / Safety 与测试分区 3
   1.1. 费曼直觉 3
   1.2. 硬核机制 3
      1.2.1. 一、逐项判定 3
      1.2.2. 二、AF：三个未闭合的问题 4
      1.2.3. 三、RI：五条条件，四种毛病 4
      1.2.4. 四、Safety：三段中唯一完全空白的一段 5
      1.2.5. 五、测试策略：分区只有骨架 5
      1.2.6. 六、与练习 1/2 的关系 7
   1.3. 陷阱与易错 7
   1.4. 关键点 7
      1.4.1. 一、优先级 7
      1.4.2. 二、自检清单（先自己回答，不急着改代码） 8
   1.5. 变式检验 8
   1.6. 同类归类 8
   1.7. Go 语言机制补充 9
   1.8. 本次操作与当前状态 10
2. ADT 测试怎么组织：operation 是外层，partition 是内层 11
   2.1. 费曼直觉 11
   2.2. 硬核机制 11
      2.2.1. 一、三个层次：suite / strategy / case 11
      2.2.2. 二、ADT 的输入不止参数 11
      2.2.3. 三、代码结构：一棵测试树 12
      2.2.4. 四、完整例子：Counter 12
      2.2.5. 五、矩阵：怎么检查没漏 14
      2.2.6. 六、序列测试放哪里 15
   2.3. 陷阱与易错 15
   2.4. 关键点 15
   2.5. 变式检验 16
   2.6. 同类归类 16
   2.7. Go 语言机制补充 16
   2.8. 本次操作与当前状态 17

Index of Listings 18

1. 0101 步骤 0 评估：AF / RI / Safety 与测试分区

本轮只评估**步骤 0**（练习 0）：`kv.go` 顶部的 `AF`、`RI`、`Safety` 三段注释，以及 `kv_test.go` 顶部的 `Testing strategy`。练习 1（`Del`）和练习 2（`Get` 未命中）的函数体不在评估范围内；它们现在的红，只作为「规格是否落地」的证据。

评估标准不是「三段标题都填了字」，而是：**另一个人只读这三段和策略，能否写出测试、判断实现是否越界，并回答「为什么」**。按这个标准，当前步骤 0 **未完成**。

1.1. 费曼直觉

把 `KV` 想成一台自动售货机：`Open` 通电，`Set` 上架，`Get` 取货，`Del` 下架。步骤 0 不是修机器，而是写四张纸：

1. **AF（Abstraction function，抽象函数）**：货架坐标怎么读成「商品表」——这是说明书；
2. **RI（Representation invariant，表示不变式）**：哪些状态算「机器正常」——这是卫生标准；
3. **Safety from rep exposure（防表示暴露）**：顾客能不能绕过柜台伸手改货架——这是防拆螺丝；
4. **Testing strategy（测试策略）**：抽查清单——先划输入空间，再挑代表。

现在的情况是：说明书里有一个没定义的词；卫生标准有五条，其中两条机器根本不可能违反、两条没人执行；防拆螺丝还没装；抽查清单只写了「有货 / 没货」一列。

> 📌 **核心要点** 步骤 0 的产出是**规格与策略**，不是能跑的代码。它没完成时，练习 1/2 的函数体即使碰巧让测试变绿，也不能证明实现正确——因为「正确」还没有定义。

1.2. 硬核机制

1.2.1. 一、逐项判定

项目	现状	判定	关键依据
`AF`	有标题、有公式	❌ 未完成	`legal key` 未定义；定义域不受 `mem` 的实际键约束；未交代 `[]byte` 到 `string` 的解释
`RI`	有标题、五条条件	⚠️ 需重写	一条类型上不可能违反；一条是 map 的语言保证；两条没有维护者；生命周期未定义
`Safety`	仍是「待填」	❌ 缺失	三段只完成两段；表示暴露是 ADT 的核心论证
测试策略	两行分区、一行序列标签	⚠️ 不完整	只覆盖 `Get` 的命中 / 未命中；缺 `Set`、`Del`、边界、序列
用例	`TestKVBasic` 能红	🟨 定位差	第 37 行断言失败、第 39 行 panic；删除后 `Get` 错了键

1.2.2. 二、AF：三个未闭合的问题

1. **抽象值是什么类型？** 是「键到值的映射」、是「有限偏函数」，还是「条目集合」？类型不同，「两个 rep 是否表示同一个抽象值」的判据就不同。
2. **定义域由谁决定？** 公式右边用了 `mem[k]`，但筛选条件是一个未定义的 `legal key`。如果它指「所有可能的键」，那么从未 `Set` 过的键也会被算进来；而 Go 里 `mem[k]` 对不存在的键返回零值 `nil`，不会报错——这会把「不存在」悄悄变成「存在且值为 nil」。先回答：`mem` 为空时 `AF` 是什么？一个从未 `Set` 过的键，应该出现在抽象值里吗？
3. **键怎么解释？** `mem` 的键是 `string`，方法参数是 `[]byte`，中间有 `string(key)` 转换。Go 的 `string` 是**只读字节序列**，不要求 UTF-8；`[]byte{0xff}` 和 `[]byte{0xc3, 0xbf}` 是两个不同的键。`AF` 要交代键按字节解释还是按字符解释，否则「两个键是否相同」没有定义。

> ⚠️ **陷阱预警** 不要用「当前 `Get` 恰好怎么返回」来反推 `AF`。`AF` 是**表示到抽象值的解释**；把 `Get` 的实现翻译一遍，得到的不是规格，是代码的另一种写法。

1.2.3. 三、RI：五条条件，四种毛病

条件	毛病	依据 / 反例	判定
`mem != nil`	生命周期没定义	零值 `KV{}` 的 `mem` 是 nil；`Open` 负责建立；`Open` 前、`Close` 后调用 `Get` / `Set` / `Del` 分别怎样？	⚠️ 需补规格
`key != nil`	类型上不可能违反	rep 的键类型是 `string`；`string` 的零值是空串，没有 nil；只有 `[]byte` 才能与 nil 比较	❌ 无效条件
`len(key) > 0`	没有维护者	`Set` 不校验；`string(nil)` 与 `string([]byte{})` 都得到空串，空键和 nil 键在 rep 里不可区分	⚠️ 规格选择未落地
`val != nil`、`len(val) > 0`	没有维护者，且与 `Get` 冲突	`Set` 原样保存；`Get` 用 `val != nil` 判断命中，存了 nil 值会被报成「未命中」	❌ 与实现冲突
`len(mem.keys) == len(mem.vals)`	语言保证，不是你的 RI	`mem` 只有一个字段，不存在 `keys` / `vals` 两个字段；map 的每个键最多对应一个值，这是 map 的定义	❌ 无效条件
缺失项	键值合法性、生命周期、同包访问	—	需补

> 📌 **核心要点** 对每条 `RI` 条件问三件事：**本类的哪段代码可能破坏它？谁负责维护？反例是什么？** 答不出来，这条条件就不该留在 `RI` 里；它是语言保证、类型保证，或者一句愿望。

⚠️ **陷阱预警** 当前 `Set` 对输入不做任何校验。只要 `RI` 写了「键非空 / 值非空」，`Set` 就必须有对应的拒绝逻辑（并定义拒绝时返回什么），否则 `RI` 只是一句愿望。反过来，如果规格决定允许空键 / nil 值， `RI` 就不该写这两条。**这是规格选择，不是实现细节**；先选，再写代码。

1.2.4. 四、Safety：三段中唯一完全空白的一段

渠道	要回答的问题	当前状态
字段 `mem`	包外能拿到吗？同包白盒测试能直接改吗？ `KV` 值复制后两个副本共享同一个 map 吗？	未填
入参	`Set` 保存的 `val` 与调用方共享底层数组吗？调用方之后修改 `val`，存储会变吗?	未填；实现是直接赋值
返回值	`Get` 返回的切片与 `mem` 共享底层数组吗？调用方修改返回值，会改到存储吗？	未填；实现是直接返回
键	`string(key)` 转换是否复制？`string` 不可变，键的风险和值一样吗？	未填

两个方向（不写代码，只选方向）：

1. **拷贝**：`Set` 存入前拷贝、`Get` 返回前拷贝；代价是每次读写都分配，但调用方怎么改都不会伤到存储；
2. **所有权约定**：规格写明「调用方不得修改传入 / 返回的切片」；代价是违约会悄悄改坏数据库。

两条路都能让 `Safety` 成立；先把选择写进规格，再写实现，别让「注释写拷贝、代码做借用」这种分裂留下来。

📌 **核心要点** `Safety` 的完成标准是**能指出每条渠道为什么安全或不安全**，不是抄一句「字段未导出」。字段未导出只回答了包外；同包测试仍能直接访问 `kv.mem`，这一点也要写进论证。

1.2.5. 五、测试策略：分区只有骨架

先看操作分类：

```
// Testing strategy
// creator: Open
// producer:
// observer: Get
// mutator: Set, Del, Close
//
// partition on key and value:
//   key map the value
//   key does not map vlue
//
// partition on operation sequence:
```

Listing 1: 当前 `Testing strategy` 摘录：两个维度、一行序列标签

问题不在「写得短」，而在**分区不完整**。当前只覆盖了 `Get` 的「命中 / 未命中」，其余操作和边界没有子域：

操作	已有分区	缺什么
`Set`	无	新键 / 已有键同值 / 已有键异值 → `updated` 分别是什么？
`Del`	无	键存在 / 键不存在 → `deleted` 分别是什么？
`Get`	命中 / 未命中	删除后再 `Get`；存了 nil 值 / 空值时的 `ok` 与 `err`
全局	无	空 key、nil key、空 val、nil val、任意二进制（含 `0x00`）、非 UTF-8
序列	只有标签	`Open` 后直接 `Get` / `Del`；`Set` → `Set`；`Set` → `Del` → `Get`；`Del` → `Del`；`Close` 后调用

分区的三条性质：**互斥**（子域不重叠）、**完整**（覆盖全部合法输入）、**非空**（每个子域能选到代表）。当前只满足了互斥；完整性和序列覆盖都还缺。

用例层面还有四个可改进点：

1. `TestKVBasic` 一条用例混了 `Open`、`Set`、`Get`、`Del`、`Close` 五个操作，失败时只能定位到行，定位不到维度；
2. 把三个条件合成一个布尔再交给 `assert.True`，失败只报「Should be true」，看不出是值错、`ok` 错还是 `err` 错；
3. `assert` 是**收集失败、继续执行**；`Del` 的 `panic` 会直接终止测试，后面的断言根本没跑。需要「失败即停」的地方用 `require`；
4. 删除 `k1` 之后，测试 `Get` 的是 `"xxx"`，不是 `"k1"`——这条断言和前面的「未命中」重复，**没有验证「删除后键真的消失」**。这正是「测试没覆盖规格」的典型：`Del` 返回 `true` 并不等于 `Get` 查不到。

> ⚠️ **陷阱预警** 分区不是「把见过的用例分类」，而是**先划输入空间，再挑代表**。先写用例再补分区，分区一定会漏掉「实现碰巧没处理」的子域；测试变绿也不能证明规格成立。

1.2.6. 六、与练习 1/2 的关系

1. 步骤 0 产出规格；练习 1（`Del`）和练习 2（`Get` 未命中）是让实现去满足规格；
2. 测试已经给了两个信号：`Get` 未命中期望 `!ok && err == nil`；`Del` 对不存在的键期望 `(false, nil)`。这两条把「未命中 / 不存在」放在了 `err == nil` 一侧；先别急着改函数体，先回答：这是测试在定义规格，还是实现需要反过来改规格？
3. 当前红基线：`kv_test.go:37` 断言失败（`Get` 未命中期望 `err == nil`），`kv.go:71` panic（`Del`）。`go vet` 通过，因为 `panic("TODO")` 是合法代码；
4. 如果 `RI` 写了「键非空 / 值非空」，练习 1/2 的实现就必须维护它；如果测试策略没覆盖某个子域，练习 1/2 变绿也不能证明规格成立。

1.3. 陷阱与易错

1. **类型事实**：`map[string][]byte` 的键是 `string`，不是 `[]byte`；`string` 不能和 `nil` 比较；`string(nil)` 合法，结果是空串。写 `RI` 前先看字段类型；
2. **语言保证**：map 的键值一一对应，不需要你写进 `RI`；`len(mem.keys) == len(mem.vals)` 是把 map 当成两个平行数组，`mem` 里根本没有 `keys` / `vals` 两个字段；
3. **零值**：`KV{}` 的 `mem` 是 nil map；nil map 读返回零值、`delete` 是 no-op、写会 panic；`Open` 负责把它变成非 nil 的空 map。`Open` 前、`Close` 后的行为要在规格里说清；
4. **别名**：slice 赋值只复制切片头，底层数组共享。`Set` 直接存 `val`、`Get` 直接返回 `val`，调用方改一下就会改到「数据库」；
5. **未命中**：Go 用 comma-ok 表达「键是否存在」；`error` 表达「操作失败」。把「没找到」塞进 `error`，会让调用方分不清「正常未命中」和「真的出错」；
6. **测试**：`assert` 收集失败继续跑，`require` 失败立即停；`panic` 会终止整个测试；合取断言会丢失定位信息；
7. **规格**：`RI` 不是愿望清单，每条都要有维护者；不要把前置条件伪装成 `RI`。

> ⚠️ **陷阱预警** 最容易骗过自己的是「测试绿了」。`TestKVBasic` 只覆盖了七个业务调用（外加一次 `Close`），`Set` 的覆盖、空值、nil 值、删除后 `Get`、`Close` 后调用都没测；绿灯只是「这几次调用没红」，不是「规格成立」。

1.4. 关键点

1.4.1. 一、优先级

级别	问题	为什么
P0	`Safety` 仍是「待填」	三段只完成两段；表示暴露是 ADT 的核心论证，且直接决定 `Set` / `Get` 要不要拷贝
P0	`RI` 与 `Set` 不一致	`RI` 禁止的输入，`Set` 没有拒绝；条件没有维护者
P0	测试没有验证「删除后键消失」	`Del` 返回 `true` 不等于 `Get` 查不到；当前删除后 `Get` 的是 `"xxx"`
P1	`AF` 的 `legal key` 未定义	定义域不受 `mem` 约束，无法判断两个 rep 是否表示同一抽象值
P1	测试分区不完整	缺 `Set` 的覆盖、`Del` 的存在性、边界、序列

P2	`producer` 空、拼写 `vlue`、用例无子域标注	格式与可维护性，但会让读者误以为漏了内容

1.4.2. 二、自检清单（先自己回答，不急着改代码）

1. `AF`：抽象值是什么类型？定义域由谁决定？`mem` 为空时是什么？从未 `Set` 的键在不在里面？键按字节还是按字符解释？
2. `RI`：每条条件能被本类代码破坏吗？谁维护？反例是什么？是语言保证、类型保证，还是规格选择？零值 `KV{}` 满足吗？`Open` 前 / `Close` 后呢？
3. `Safety`：字段、入参、返回值、键四条渠道，各自为什么安全或不安全？选拷贝还是所有权约定？
4. 测试：每个操作的维度列全了吗？边界在哪？序列覆盖了吗？每个子域有代表用例吗？删除后 `Get` 的是哪个键？
5. 一致性：`RI` 禁止的输入，`Set` 拒绝了吗？测试覆盖了吗？测试的断言和你的规格说的是同一件事吗？

> 📌 **核心要点** 完成步骤 0 的标志：把这三段和策略交给另一个 Go 程序员，他**不需要看你的实现**，就能写出覆盖主要子域的测试，并能判断某个 `Set` / `Get` / `Del` 实现有没有越界。做不到，就还没完成。

1.5. 变式检验

不要求现在写代码；用来检验「你是理解了规格，还是背了三个标题」：

1. 把 `KV` 换成 `map[string]int` 的计数器（`Increment` / `Get` / `Delete`）：`AF`、`RI`、`Safety` 分别哪里变？`Delete` 的返回值还能表达「删没删到」吗？
2. 把值换成 `*bytes.Buffer`：`Safety` 的问题会变大还是变小？为什么？
3. 给 `KV` 加一个 `log` 字段（0103 的方向）：**跨字段不变量**第一次真正出现；`AF` 和 `RI` 要怎么变？`Open` 要负责什么？
4. 把 `Get` 的返回值改成 `(val []byte, found bool)`：未命中语义怎么表达？和现在的 `(val, ok, err)` 比，哪个更难误用？
5. 如果想直接用 `[]byte` 做 map 的键：为什么 Go 不允许？可以怎么绕？绕法会带来什么新问题？

1.6. 同类归类

概念	材料	本步证据	判定
C05 规格	6.102 R04 / 6.031 R06	未命中、空值、nil、错误的语义	🟨
C08 ADT	6.102 R04 / 6.031 R06	表示独立、`AF` / `RI`	🟨
C09 AF / RI / Safety	6.102 R04 / 6.031 R07	三段注释	❌ / ⚠️

C02 测试与分区	6.102 R02 / 6.031 R03	`Testing strategy` 与 `TestKVBasic`	🟨

一句话：这一步的主概念是 **C05 规格**，辅 **C08 ADT** 与 **C09 AF/RI/Safety**；测试策略是 **C02** 的应用。不要把这一步当成「写三个注释」，它是在定义「什么叫对」。

1.7. Go 语言机制补充

术语表（首次出现的一句话定义 + 最小例子）：

术语	一句话定义	最小例子
map	Go 内置哈希表，键必须可比较	`m := map[string]int{}`；`[]byte` 不能做键
string	只读字节序列，不要求 UTF-8	`string([]byte{0xff})` 合法
nil map	零值 map：读安全、写 panic、`delete` no-op	`var m map[string]int`
slice 别名	赋值只复制切片头，底层数组共享	`b := a; b[0] = 1` 会改到 `a`
comma-ok	两值赋值判断键是否存在	`v, ok := m[k]`
`assert` vs `require`	testify 的两种断言：继续 vs 立即停止	`require.NoError(t, err)`

两段最小实验（把片段放进 `main` 或测试函数里跑）：

```
var m map[string][]byte        // 零值：nil map
_ = m[string([]byte{})]        // 读：返回 nil，不 panic
delete(m, string([]byte{}))    // delete: no-op
// m[string([]byte{})] = nil   // 写：panic
```

Listing 2: nil map 的读、删、写三种行为

写 nil map 的 panic 信息是 `assignment to entry in nil map`；这是运行时 panic，不是 `error`。

```
key := []byte{0x6b, 0x31}
s := string(key)       // 得到不可变 string
key[0] = 0x78          // 改原切片，不影响 s

val := []byte{1, 2, 3}
stored := val          // 只复制切片头，底层数组共享
stored[0] = 9          // val[0] 也变成 9
```

Listing 3: string 不可变；slice 赋值共享底层数组

> ⚠️ **陷阱预警** `string` 是字节序列，不是「文本」。空串、任意字节（包括 `0x00`、非 UTF-8）都可以是合法键——这是**语言事实**；但「你的 `KV` 允不允许空键」是**规格选择**，两者别混。

相关命令（本机已验证可用）：

1. `go doc builtin` —— 查内建类型与函数的官方说明；
2. `go doc bytes.Clone` —— 查切片拷贝的官方 API；
3. `go doc github.com/stretchr/testify/require` —— 查 `require` 与 `assert` 的区别；
4. `go doc -src strings.Clone` —— 看标准库怎么做「明确的拷贝」。

1.8. 本次操作与当前状态

1. **检查范围**：`internal/db/kv.go`（`AF`、`RI`、`Safety` 与五个方法）、`internal/db/kv_test.go`（`Testing strategy` 与 `TestKVBasic`）、`git diff`；没有改 `kv.go` / `kv_test.go`，也没有给出 `AF` / `RI` / `Safety` 的修正答案；
2. **验证命令与结果**：
   - `go test -count=1 ./internal/db/ -run TestKVBasic -v` → FAIL：`kv_test.go:37` 断言失败（`Get` 未命中期望 `err == nil`），`kv.go:71` panic（`Del`）；
   - `go vet ./internal/db/` → 通过；
   - `go doc ./internal/db KV` → 能读到三段注释，说明它们就是公开契约的落点；
   - `typst compile 2026-10-31.typ` → 零错误，PDF 与 `.typ` 同级；
3. **本轮结论**：步骤 0 **未完成**—— `Safety` 空白；`RI` 有五条但两条无效、两条无维护者、一条生命周期未定义；`AF` 的定义域依赖未定义的 `legal key`；测试策略只覆盖 `Get` 的命中 / 未命中，且删除后 `Get` 错了键；
4. **下一步**：先回答「自检清单」，修改三段注释与测试策略；改完再进入练习 1（`Del`）和练习 2（`Get` 未命中）。不要先改实现——规格没定，实现只能靠猜。

2. ADT 测试怎么组织：operation 是外层，partition 是内层

先把结论说清楚：**operation 和 partition 不是两根要同时穷举的轴，而是两层循环。**外层是 operation（测哪个方法），内层是 partition（这个方法在这个状态下有哪些等价输入）。你感到迷茫，多半是因为把测试策略里的 `partition on operation` 当成了「operation 也是一个输入维度」——它其实是**测试覆盖清单**，不是内层分区。

2.1. 费曼直觉

把测试套件想成一棵树：

1. **第一层（operation）**：考哪几门课——`TestKVGet`、`TestKVSet`、`TestKVDel`、`TestKVSequence`；
2. **第二层（partition）**：每门课出哪些题——`Get` 在「键存在 / 不存在 / 删除后」各考一次；
3. **隐式输入（`this` 状态）**：考前你掌握到什么程度——空 `KV`、已有 `k1`、已删除 `k1`。

考试不会要求你「同时想所有科目和所有题型」；你先定科目，再给每科出题。测试也一样：先按 operation 分组，再在组内按 partition 选输入。

> 📌 **核心要点** `operation` 回答「测谁」，`partition` 回答「用哪些输入测」。前者是测试的组织单位，后者是输入空间的划分方法；它们不在同一层，不需要在一个脑子里同时穷举。

2.2. 硬核机制

2.2.1. 一、三个层次：suite / strategy / case

层次	单位	写在哪	例子
suite	operation	测试函数 / 子测试组	`TestCounterInc`、`TestCounterValue`
strategy	operation 的输入维度	文件顶部或函数注释	`Inc`：状态 × `delta` 符号
case	一个代表	`t.Run` 子测试名	`inc/from-nonzero`

关键澄清：`partition on operation` 这一行属于 **suite 层**，它回答「每个 operation 都测到了吗」；它**不是**「把 operation 当成一个输入维度」。真正的 partition 在 **case 层**：每个 operation 内部的参数 × 状态。

2.2.2. 二、ADT 的输入不止参数

输入来源	普通函数	ADT 方法	例子
参数	`f(x)`	`kv.Get(k)`	`k`
接收者状态	无	`kv` 的抽象状态	空 / 有 `k1` / 已删除
调用历史	无	之前的 `Set` / `Del`	`Set` → `Get`
生命周期	可能	`Open` 前 / `Close` 后	未打开 / 已关闭

所以 ADT 测试的每个 operation 组内部，至少有两维：**状态** × **参数**。状态由 creator / producer 构造，断言由 observer 读。这个「状态也是输入」的事实，是 ADT 测试和普通函数测试的唯一本质区别。

2.2.3. 三、代码结构：一棵测试树

```
TestKVGet
  /<state × key>
TestKVSet
  /<state × key × value>
TestKVDel
  /<state × key>
TestKVSequence
  /<operation sequence>
```

Listing 4: 测试树：第一层是 operation，第二层是 partition 代表；尖括号由你填

`TestKVGet` 里只放 `Get` 的用例；`TestKVSet` 里只放 `Set` 的用例；跨操作的交互放 `TestKVSequence`。这不是「多写几个函数」，而是让失败信息直接告诉你「哪个 operation 的哪个分区红了」。

2.2.4. 四、完整例子：Counter

场景：一个页面访问计数器，`Inc(delta)` 累加，`Value()` 读当前值，`Reset()` 在统计窗口结束时清零。

约束从哪来：`Inc` 的 `delta` 必须非负——这是规格的前置条件；负数会让计数倒退，监控阈值和告警会把「-1」当成异常。`Reset` 是 mutator，必须在窗口切换时把值归零。

不遵守会怎样：如果 `Inc` 不声明非负前置条件，调用方传负数，`Value` 可能小于 0；如果 `Reset` 只测「调用不 panic」，没测 `Value` 归零，窗口切换后旧值会残留，统计翻倍。

与项目的关系：`Counter` 是单值 ADT，`KV` 是多键 ADT；测试组织方式相同——operation 分组 + 状态 / 参数分区 + observer 断言。差别是 `KV` 多一个「键」维度和一个「删除后状态」；不要把 `Counter` 的分区直接照搬成 `KV` 的答案。

```go
package counter

// Counter 是一个单值计数器；Inc 的 delta 必须非负（前置条件）。
type Counter struct {
  n int
}

func (c *Counter) Inc(delta int) {
  c.n += delta
}

func (c *Counter) Value() int {
  return c.n
}

func (c *Counter) Reset() {
  c.n = 0
}
```

Listing 5: counter.go：creator 是零值，mutator 是 `Inc` / `Reset`，observer 是 `Value`

```go
package counter

import `testing` // 等价于双引号 import 写法；这里用 raw string 只是排版

func newCounter(t *testing.T) *Counter {
  t.Helper()
  return &Counter{}
}

func TestCounterInc(t *testing.T) {
  tests := []struct {
    name  string
    setup func(*Counter)
    delta int
    want  int
  }{
    {`from-zero`, func(c *Counter) {}, 1, 1},
    {`from-nonzero`, func(c *Counter) { c.Inc(5) }, 3, 8},
    {`zero-delta`, func(c *Counter) { c.Inc(5) }, 0, 5},
  }
  for _, tt := range tests {
    t.Run(tt.name, func(t *testing.T) {
      c := newCounter(t)
      tt.setup(c)
      c.Inc(tt.delta)
      if got := c.Value(); got != tt.want {
        t.Fatalf(`Value() = %d, want %d`, got, tt.want)
      }
    })
  }
}
```

Listing 6: `counter_test.go` 上半：文件头、夹具与 `TestCounterInc`

```go
func TestCounterValue(t *testing.T) {
  c := newCounter(t)
  if got := c.Value(); got != 0 {
    t.Fatalf(`initial Value() = %d, want 0`, got)
  }
  c.Inc(2)
  if got := c.Value(); got != 2 {
    t.Fatalf(`after Inc(2), Value() = %d, want 2`, got)
  }
}

func TestCounterReset(t *testing.T) {
  c := newCounter(t)
  c.Inc(7)
  c.Reset()
  if got := c.Value(); got != 0 {
    t.Fatalf(`after Reset(), Value() = %d, want 0`, got)
  }
}

func TestCounterSequence(t *testing.T) {
  c := newCounter(t)
  c.Inc(3)
  c.Inc(4)
  c.Reset()
  c.Inc(1)
  if got := c.Value(); got != 1 {
    t.Fatalf(`after Inc,Inc,Reset,Inc, Value() = %d, want 1`, got)
  }
}
```

Listing 7: `counter_test.go` 下半：`Value` / `Reset` / `Sequence` 三组，接在上一个 listing 之后

2.2.5. 五、矩阵：怎么检查没漏

测试组	初始状态	有值状态	参数边界
`TestCounterInc`	`from-zero`	`from-nonzero`	`zero-delta`
`TestCounterValue`	`initial`	`after-inc`	—
`TestCounterReset`	—	`after-inc`	—
`TestCounterSequence`	序列：`Inc` → `Inc` → `Reset` → `Inc`	—	—

operation 是行，partition 是列；格子是代表用例。空格子要么补用例，要么写「不适用」。这张表不是「把 operation 当维度」，而是「用 operation 定位测试组，用 partition 检查覆盖」。

逐段解释：

1. `TestCounterInc` 的 `from-zero` / `from-nonzero` 是**状态分区**；`zero-delta` 是**参数边界**。三者都在同一个 operation 组里，用 `setup` 构造不同的 `this` 状态；

2. `TestCounterValue` 的 `initial` / `after-inc` 是状态分区；`Value` 在这里既是被测，也是其他测试的断言工具——observer 的双重角色是 ADT 测试的常态；
3. `TestCounterReset` 只测 `Reset`，`Inc(7)` 在 setup 里负责构造「有值」状态；
4. `TestCounterSequence` 是序列组，不混进上面三组；它测的是操作之间的状态转换，不是某一个方法的参数空间。

2.2.6. 六、序列测试放哪里

测试组	覆盖什么	例子
单操作组	该方法在各类状态下的行为	`TestCounterInc`、`TestCounterReset`
序列组	跨操作的状态转换	`Inc` → `Inc` → `Reset` → `Inc`

序列组仍然用 observer 断言；不要为了「测序列」直接读 rep。`Set` → `Get`、`Set` → `Del` → `Get`、`Del` → `Del` 这些顺序，放在序列组里，不要塞进每个 operation 的测试。

2.3. 陷阱与易错

误区	症状	修正
把 suite 清单当维度	想在一个函数里穷举多个 operation	operation 属于 suite 层；内层 partition 只管参数 × 状态
每个 operation 都从空状态开始	漏掉「非空 / 已删除」状态	用 setup 构造前置状态
直接读 `kv.mem` 断言	测的是实现，不是规格	用 `Get` 等 observer 断言
只测单操作	漏掉交互 bug	加序列组
把 partition 写成用例列表	分区不完整	先划输入空间，再选代表

> ⚠️ **陷阱预警** 不要为了「按 operation 分组」就把共享的 `setup` / 断言复制五遍。共享的是夹具（`newKV`、`put`），分开的是测试组；DRY 和分层不冲突。反过来，也不要把五个 operation 塞进一个 `TestKVBasic`——那样失败时你不知道是哪个 operation 的哪个分区红了。

2.4. 关键点

1. **operation 是外层**：测试树第一层按方法分组；每个组只关心自己的契约；
2. **partition 是内层**：组内按「状态 × 参数」划分子域，每个子域选代表；
3. **序列单独一组**：跨操作的交互不属于任何一个 operation 的参数空间；
4. **observer 负责断言**：用 `Get` / `Value` 读抽象值，不要直接读 rep。

自检清单：

1. 每个 operation 至少有一个测试组吗？creator / producer / observer / mutator 都覆盖了吗？
2. 每个组内，**状态维度**和**参数维度**都列了吗？边界（空、零、nil、重复）在哪？

3. 每个子域有代表用例吗？子测试名能看出它覆盖哪个子域吗？
4. 断言用的是 observer 还是 rep？如果是 rep，为什么必须白盒？
5. 序列组覆盖了哪些跨操作交互？`Set` → `Del` → `Get` 这类顺序有没有漏？
6. 覆盖矩阵有没有空格子？空格子是「不适用」还是「漏了」？

> 📌 **核心要点** 完成标准：把测试树和策略注释交给另一个 Go 程序员，他**不需要看实现**，就能补出一个覆盖主要子域的用例，并说清这个用例属于哪个 operation 的哪个分区。

2.5. 变式检验

1. 把 `Counter` 换成 `Buffer`（固定容量队列，`Put` / `Get` / `Len`）：operation 分组怎么变？状态分区多出「空 / 满 / 中间」哪几格？
2. 把 `Get` 的返回值从 `(val, ok)` 改成 `(val, error)`：未命中的分区还在，但断言从 `ok == false` 变成什么？测试树要改哪一层？
3. 给 `KV` 加 `Close` 生命周期：多出哪一组测试？「`Close` 后调用 `Get`」属于 operation 分组还是序列组？
4. 把 ADT 换成不可变的 `RatNum`（`Add` / `Mul` / `Equals`）：没有 mutator，operation 分组和状态分区分别怎么变？
5. 加并发：`go test -race` 下，「并发调用」是新的 operation 组，还是每个 mutator 组里的一个维度？先想，不写代码。

2.6. 同类归类

概念	材料	本文落点
C02 测试分区	6.102 R02 / 6.031 R03	partition 是内层：参数 × 状态
C08 ADT	6.102 R04 / 6.031 R06	operation 是外层：测试组按方法分
C05 规格	6.102 R04 / 6.031 R06	状态分区来自规格（哪些状态合法）
C09 AF / RI	6.102 R04 / 6.031 R07	状态由 `AF` / `RI` 定义，测试只观察抽象值

与已有笔记的关系：`note/2026-10-26.typ` 的「操作分类在哪个阶段」和「Testing an abstract data type 与五步」已经讲过「外层框架 + 内层五步」；本文补的是**代码结构怎么落地**，以及 `partition on operation` 这一行的歧义。`note/2026-10-17.typ` 的表 1（操作分类）和表 4（测试分区）可以作为填 `KV` 矩阵的参照。

2.7. Go 语言机制补充

术语表（首次出现的一句话定义 + 最小例子）：

术语	一句话定义	最小例子
子测试	`t.Run` 创建的命名测试，可单独运行	`t.Run(from-zero, f)`
表驱动	把用例写成切片，循环跑	`tests := []struct{ name string }`

夹具	测试前构造的状态	`newCounter(t)`
黑盒 / 白盒	只看规格 / 也看实现	`counter_test` vs `counter`
覆盖率	代码被执行的比例	`go test -cover`

`t.Run` 、 `t.Helper` 、 `t.Cleanup` 的分工：

1. `t.Run(name, f)` ：创建子测试；名字会成为 `-run` 的路径（ `TestCounterInc/from-zero` ），失败时能定位到具体子域；
2. `t.Helper()` ：把当前函数标记为 helper；断言失败时行号指向**调用 helper 的地方**，而不是 helper 内部；
3. `t.Cleanup(f)` ：注册测试结束时的清理，按后进先出执行；比在 helper 里写 `defer` 更合适——helper 返回时 `defer` 就执行了，而 `t.Cleanup` 会等到整个测试（含子测试）结束。

相关命令（本机已验证可用）：

1. `go doc testing.T.Run` —— 子测试的官方说明；
2. `go doc testing.T.Cleanup` —— 清理函数的执行时机；
3. `go help test` —— `go test` 的完整用法；
4. `go test -run 'TestCounter/from-zero' -v` —— 只跑一个子测试；
5. `go test -cover` —— 看代码覆盖率（注意：覆盖率高不等于分区完整）。

> ⚠️ **陷阱预警** 同包测试（ `package counter` ）可以直接读私有字段，这很方便，但 ADT 测试优先用公开 API 的 observer 断言；白盒只用于 `checkRep` 或内部不变量。直接读 rep 的测试会在换实现时全部失效。

2.8. 本次操作与当前状态

1. **检查范围**： `internal/db/kv.go` 、 `internal/db/kv_test.go` （只读，未修改）；新增本节的方法论说明；没有替 `KV` 填具体分区，也没有改练习 0 的注释；
2. **验证命令与结果**：
   - `go doc testing.T.Run` 、 `go doc testing.T.Cleanup` → 可用； `Counter` 示例已在临时目录用 `go test` 验证可编译（标准库 `testing` ，无第三方依赖）；
   - `go vet ./...` 、 `go build ./...` → 通过；
   - `typst compile 2026-10-31.typ` → 零错误，PDF 与 `.typ` 同级；
3. **本轮结论**： operation 和 partition 是两层，不是两根需要同时穷举的轴； `partition on operation` 是 suite 层的覆盖清单，不是内层输入维度；内层永远按「状态 × 参数」分区，序列单独成组；
4. **下一步**：按 `TestKVGet` / `TestKVSet` / `TestKVDel` / `TestKVSequence` 四组给 `KV` 建测试树，先填每组的「状态 × 参数」维度，再选代表；不要把所有 operation 塞进一个测试函数。

Index of Listings

Listing 1 当前 `Testing strategy` 摘录：两个维度、一行序列标签 6
Listing 2 nil map 的读、删、写三种行为 9
Listing 3 string 不可变；slice 赋值共享底层数组 9
Listing 4 测试树：第一层是 operation，第二层是 partition 代表；尖括号由你填 12
Listing 5 counter.go：creator 是零值，mutator 是 `Inc` / `Reset`，observer 是 `Value` .. 13
Listing 6 `counter_test.go` 上半：文件头、夹具与 `TestCounterInc` 13
Listing 7 `counter_test.go` 下半：`Value` / `Reset` / `Sequence` 三组，接在上一个 listing 之后 14